Fichas bibliográficas: fundamentos de sistemas distribuidos y arquitecturas distribuidas

Manotoa Alava Carlos Alexander

11 de Noviembre del 2026

FUENTE 1: MONOLITHIC VS. MICROSERVICE ARCHITECTURE: A PERFORMANCE AND SCALABILITY EVALUATION

Referencia IEEE: G. Blinowski et al., "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

Este artículo es una comparación del rendimiento y la escalabilidad que presentan las arquitecturas monolíticas frente a microservicios. El problema es que pequeñas empresas migran a microservicios esperando beneficios similares a lo que reportan las grades empresas tecnológicas, pero para sistemas sin miles de usuarios concurrentes y escalabilidad vertical, estos beneficios no han sido investigados a profundidad. El método consiste en implementar una aplicación web referencial en 4 versiones (Monolito vs microservicios y Java vs C#) y realizar experimentos controlados en un entorno local, en Azure Spring Cloud y en Azure App Service, usando JMeter.

Los hallazgos clave son:

- En una sola máquina monolito rinde mejor
- El escalado vertical es más rentable que el horizontal en Azure
- Escalar más allá de ciertos números de instancias degrada el rendimiento
- La tecnología de implementación no afecta notablemente la escalabilidad.

En conclusión, el artículo muestra que los microservicios no son la mejor opción para todos los contextos.

Dato concreto: En la sección V.Results. A. Local Environment, el estudio reporta que monolito fue capaz de manejar en promedio más del doble de solicitudes por segundo que la versión de microservicios en .NET para el servicio de City, y 1.37 veces más en Java.

Nota personal: Esta fuente permite comprender que la decisión entre monolito y microservicios depende del contexto, para sistemas pequeños sin miles de usuarios concurrentes, un monolito con escalabilidad vertical puede ser más eficiente y económico que migrar a

microservicios. La usaré en la sección de análisis de arquitectura de TA-IND-01 para justificar que la elección entre monolítico y microservicios debe tomarse en el contexto y mediciones reales, no en tendencias o popularidad.

FUENTE 2: DEPLOYMENT AND COMMUNICATION PATTERNS IN MICROSERVICE ARCHITECTURES: A SYSTEMATIC LITERATURE REVIEW

Referencia IEEE: I. Karabey Aksakalli y T. Çelik, "Deployment and communication patterns in microservice architectures: A systematic literature review," *Journal of Systems and Software*, vol. 180, pág. 111 014, 2021. DOI: 10.1016/j.jss.2021.111014

Esta fuente revisa la necesidad de identificar y describir los diferentes enfoques de despliegue y patrones de comunicación empleados en arquitecturas de microservicios, así como los problemas encontrados y sus resoluciones. Para ello se realizó una revisión sistemática de literatura con protocolo de Kitchenham. La fuente revisa 239 artículos y selecciona 38 artículos mediante criterios de inclusión/exclusión y snowballing. Los resultados identifican tres enfoques de despliegue:

- Serverless
- Instancia de servicio por VM
- Instancia de servicio por contenedor

También identifican siete patrones de comunicación:

- Síncrona
- Publish/subscribe
- Combinación HTTP y cola de mensajes
- Middleware orientado a mensajes
- Asíncrona
- Punto a punto
- Protocolos binarios

Además, también reporta ocho retos de despliegue, seite de comunicación y nueve direcciones de investigación.

Dato concreto: Se revisaron 239 artículos de los cuales se seleccionaron 38. Asimismo, se identifican tres tipos de enfoques de despliegue y siete patrones de comificación.

Nota personal: Esta fuente me ayuda a comprender que no basta con distribuir un sistema grande; la forma en que cada uno se instala y comunica con los otros son decisiones claves para que el sistema funcione correctamente. En TA-IND-01 la usaré en la sección de Arquitectura y Diseño para justificar la selección de contenedores y patrones de comunicación según los requisitos, y para revisar temas como costos de despliegue, tolerancia a fallos y balanceo de cargas.

FUENTE 3: CONTAINERIZED MICROSERVICES: A SURVEY OF RESOURCE MANAGEMENT FRAMEWORKS

Referencia IEEE: L. M. Al Qassem et al., "Containerized Microservices: A Survey of Resource Management Frameworks," *IEEE Transactions on Network and Service Management*, vol. 21, n.º 4, págs. 3775-3796, 2024. DOI: 10.1109/TNSM.2024.3388633

Este artículo aborda el problema de la gestión eficiente de recursos en arquitecturas de microservicios contenerizados en la nube, donde la asignación dinámica, el cumplimiento de los SLA y el consumo energético son difíciles de optimizar. Para entender cómo se manejan los recursos, se seleccionaron 65 artículos y se clasificaron en tres tipos:

- Reactivos: Los que responden en tiempo real.
- Predictivos: Los que buscan anticiparse al futuro
- Híbridos: Los que combinan ambos métodos

En los resultados se observa que los tipos reactivos son los más usados, mientras que los predictivos e híbridos no son tan frecuentes. Además, el artículo destaca la importancia de cumplir y simplificar los acuerdos de calidad del servicio (SLA) y de compartir recursos entre microservicios según la carga de trabajo. La conclusión es que hacen falta soluciones más seguras y robustas que puedan soportar el flujo de trabajo real en una empresa.

Dato concreto: La tabla IV de la página 16, indica que los enfoques reactivos son los más adoptados con un 57,69 %, frente a los predictivos con un 30,7 % e híbridos con un 11,5 %.

Nota personal: Esta fuente ayuda a comprender que la gestión contenerizada exige decidir entre un escalado reactivo, predictivo o híbrido, y equilibrar SLA, costos y energía. La usaré en TA-IND-01, en la sección de "despliegue y operaciones", para justificar una estrategia de escalado.

FUENTE 4: VISUALIZING ANTI-PATTERNS IN MICROSERVICES AT RUNTIME: A SYSTEMATIC MAPPING STUDY

Referencia IEEE: G. Parker et al., "Visualizing Anti-Patterns in Microservices at Runtime: A Systematic Mapping Study," *IEEE Access*, vol. 11, págs. 4434-4442, 2023. DOI: 10.1109/ACCESS.2023.3236165

El estudio aborda la dificultad de detectar y visualizar antipatrones en microservicios desde una perspectiva dinámica, porque muchos aparecen en ejecución, no solo en revisiones estáticas. Además, pocos trabajos combinan detección y visualización. Para entender el problema, se realizó un mapeo sistemático sobre detección y visualización de antipatrones desde una perspectiva dinámica. Los resultados muestran que existen muchos métodos para detectar antipatrones en tiempo de ejecución. Por ejemplo:

- El procesamiento de eventos complejos

- El análisis de grafos y la revisión de logs o trazas

También existen herramientas para visualizar la arquitectura de un sistema de microservicios. Sin embargo, no se encontró una herramienta única que tenga ambas capacidades (detectar y visualizar), lo que comúnmente se hace es combinar herramientas de detección con herramientas de visualización. Los antipatrones más reportados y fáciles de representar mediante grafos de dependencias de servicios, son las dependencias cíclicas. La conclusión es que combinar detección en ejecución con visualización ayuda a identificar y corregir antipatrones, pero todavía no se ha logrado una solución robusta que haga ambas cosas.

Dato concreto: La Tabla 2, de la página 6 muestra que las dependencias cíclicas son el antipatrón más frecuentemente detectado y visualizado.

Nota personal: Esta fuente me ayuda a comprender que la detección y la visualización son dos actividades distintas. La usaré en TA-IND-01, en la sección de "observabilidad y análisis dinámico", para justificar la importancia de seguir el recorrido de una petición (las trazas distribuidas) y ver cómo se conectan los servicios (visualización de dependencias).

FUENTE 5: ARCHITECTURE MIGRATION FROM MONOLITHIC TO MICROSERVICES: DEVELOPING READINESS CRITERIA

Referencia IEEE: P. I. Habib et al., "Architecture Migration From Monolithic to Microservices: Developing Readiness Criteria," *IEEE Access*, vol. 12, págs. 194 630-194 645, 2024. DOI: 10.1109/ACCESS.2024.3504848

Este artículo trata sobre la dificultad de decidir y ejecutar la migración de una arquitectura monolítica a microservicios, ya que muchas organizaciones migran por seguir las tendencias, sin una comprensión real y topándose con problemas de planificación, infraestructura, descomposición de servicios, gestión de datos, seguridad y monitoreo. El estudio busca desarrollar criterios de preparación que ayuden a evaluar si una organización está lista antes de comenzar a realizar el cambio. Los resultados muestran que deben considerarse factores organizacionales, humanos, tecnológicos y del entorno. Algunos de los criterios que se destacan son el compromiso de los líderes, la capacidad de los programadores, la satisfacción de los usuarios, la calidad de la información y la seguridad. En conclusión, el documento es una guía para que las organizaciones tomen mejores decisiones, reduzcan riesgos y no migren solo por moda.

Dato concreto: En la sección IV, numeral D, el ANP reporta que los criterios que más influyen son:

- Organizacionales con un 0,33
- Humano con un 0,28
- Entorno con un 0,20
- Tecnológico con un 0,17

Además, la Tabla 13 de la página 14 registra un puntaje total de 64,23 para el caso BOS.

Nota personal: Esta fuente me permite entender que la migración a una arquitectura de microservicios debe evaluarse con criterios humanos, organizacionales, de ambiente y técnicos, no solo por tendencias o modas. La usaré en el apartado de "Análisis de Viabilidad" del TA-IND-01 para ver si realmente si los proyectos cuentan con compromiso, capacidades necesarias, usuarios satisfechos y condiciones técnicas adecuadas antes de pasar a microservicios.

Declaración de la IA

Durante la elaboración de este trabajo, se uso la IA con el propósito de corregir la ortografía, la gramática y dar formato de los documentos. El contenido de los resumenes fue hecho manualmente.

Referencias

[1] G. Blinowski, A. Ojdowska y A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

[2] I. Karabey Aksakalli y T. Çelik, "Deployment and communication patterns in microservice architectures: A systematic literature review," *Journal of Systems and Software*, vol. 180, pág. 111 014, 2021. DOI: 10.1016/j.jss.2021.111014

[3] L. M. Al Qassem, T. Stouraitis, E. Damiani e I. M. Elfadel, "Containerized Microservices: A Survey of Resource Management Frameworks," *IEEE Transactions on Network and Service Management*, vol. 21, n.º 4, págs. 3775-3796, 2024. DOI: 10.1109/TNSM.2024.3388633

[4] G. Parker, M. Kim y T. Cerny, "Visualizing Anti-Patterns in Microservices at Runtime: A Systematic Mapping Study," *IEEE Access*, vol. 11, págs. 4434-4442, 2023. DOI: 10.1109/ACCESS.2023.3236165

[5] P. I. Habib, F. A. Murdha y M. F. Al-Rahmawy, "Architecture Migration From Monolithic to Microservices: Developing Readiness Criteria," *IEEE Access*, vol. 12, págs. 194 630-194 645, 2024. DOI: 10.1109/ACCESS.2024.3504848